ADVANCED SDK FPGA IP VERSION 6.3

12/20/2025

Table of Contents

1 NDI FPGA Reference Design 4
1.1 Quick Start 4
1.2 Directory structure: 4
1.3 Design Overview 4
1.4 Theory of Operation 5
1.4.1 Common Encode and Decode Base Functionality 5
1.4.2 NDI Encode Operation 7
1.4.3 NDI Decode Operation 11
1.4.4 Supported Video Formats 13
1.5 Rebuilding from source 16
1.5.1 FPGA hardware design 16
1.5.2 Boot loader, Linux kernel, and device-tree 16
1.5.3 Application software 16
1.5.4 Bootable uSD image 16
2 FPGA Projects 17
2.1 Directory structure 17
2.2 Build Dependencies 17
2.2.1 Xilinx 17
2.2.2 Altera 18
2.3 Rebuilding 19
2.3.1 Xilinx 19
2.3.2 Altera 19
2.4 Known Issues and Limitations: 19
2.5 NDI IP Files and Use 20

2.5.1 IP Core Configuration 20
2.5.2 NDI Encoder 20
2.5.3 NDI Decoder 21
2.5.4 SDRAM Bus Interfaces 21
2.5.5 Full vs. Lite Encode Designs 22
2.6 HDMI Input Logic 22
2.6.1 Agilex-7 SoC DevKit 22
2.6.2 Arria-10 SoC DevKit 22
2.6.3 Zynq 7000 HDMI Input 23
2.6.4 ZCU104 HDMI Input 23
2.7 HDMI Output Logic 25
2.7.1 Arria-10 SoC DevKit 25
2.7.2 Zybo-Z7-20 HDMI Output 26
2.7.3 ZCU104 HDMI Output 26
2.7.4 SoCKit-Dec 28
2.8 MIPI CSI Input Logic 28
2.8.1 Xilinx Kria KV260 28
3 C++ Application Code 29
3.1 Directory structure 29
3.2 Getting Started 29
3.2.1 Prerequisites 29
3.2.2 Building and Installing 29
3.3 ndi_encode 30
3.3.1 Usage 30
3.3.2 Run Time Commands 30
3.3.3 Command Line Options 30
3.4 ndi_decode 33
3.4.1 Usage 33
3.4.2 Command Line Options 33
3.5 ndi_reg 35
3.5.1 Register Map 35
4 Linux Kernel and Boot Loader 36
4.1 Altera 36
4.1.1 Directory Structure 36
4.1.2 Build Requirements 36
4.1.3 Building an Existing Project 36

4.2 Xilinx 37
4.2.1 Open Source Linux (OSL) Flow 37
4.2.2 Petalinux Flow 40
5 uSD Image Builder 41
5.1 Build Dependencies 41
5.2 Build an image 42
5.3 Rebuild an image 42
6 Changelog 44
6.1 FPGA Projects Changelog 46
6.2 C++ Application Code Changelog 50
6.3 Linux Kernel and Boot Loader Changelog 53
6.4 uSD Image Builder Changelog 55

1 NDI FPGA REFERENCE DESIGN

The NDI Advanced SDK contains working example designs intended to assist in using the NDI Advanced software SDK and the NDI FPGA IP cores.

Building a working embedded NDI design requires wide-ranging expertise and these example designs are provided in an attempt to make this process more accessible. Prebuilt uSD card images are available for the supported platforms and details are provided regarding how to rebuild each required piece from scratch.

1.1 QUICK START

1. Obtain one of the supported development boards:
    - Altera Agilex-7 I-Series Transceiver-SoC Devkit
    - Altera Arria-10 SoC Devkit
    - Digilent Zybo-Z7-20
    - Digilent Arty-Z7-20 (partial support, the Zybo-Z7-20 is preferred)
    - Xilinx ZCU104
    - Xilinx Kria KV260 (partial support, the ZCU104 is preferred)
2. Boot your board using a prebuilt image

    See the `FPGA Quick Start Guide.pdf` file for details on obtaining the uSD image, writing it to a uSD card, and using it with one of the supported platforms.

1.2 DIRECTORY STRUCTURE:

The FPGA reference design is organized into the following files and subdirectories under the `fpga_reference_design/` directory:

Filename	Contents
`README.md`	High level overview of the provided example projects
`README.uSD.md`	Details on using the prebuilt uSD images
`CHANGELOG.md`	List of notable changes
`src/cpp`	Software source code and example projects
`src/fpga`	Hardware design files and example projects
`linux_kernel`	Projects to build kernel and boot loader
`os_uSD`	Scripts to generate root filesystem and bootable uSD images

1.3 DESIGN OVERVIEW

There are a lot of different pieces required to create a working example system:

- FPGA hardware design
- Boot loader
- Linux kernel
- Device-tree
- Linux root file system
- Application software
- Bootable image including all of the above

While each of these components is important and necessary, not all of them have to be customized in order to create a custom project. The three components that need to be customized for almost any project are the actual FPGA hardware, the application software, and the device tree. The Linux kernel and boot loader can typically be used without customizations and there are several options for creating a Linux root file system.

This example design uses vendor specific processes to generate the boot loader, the Linux kernel, and most of the device tree. The root file system is a standard Debian install. The FPGA logic and application software are custom, and some device-tree customizations are required to support the custom FPGA hardware. Each of these components is discussed in more detail below.

1.4 THEORY OF OPERATION

Embedded systems require interaction between hardware and software. This is a complex process even on small micro-controllers, and is even more so on systems running a high-level OS such as Linux. This example design was created with the intent to make the interaction between hardware and software as simple as possible to implement and modify, with no need to write kernel-space code.

1.4.1 COMMON ENCODE AND DECODE BASE FUNCTIONALITY

1.4.1.1 LOW LEVEL DETAILS AND DEVICE-TREE

In embedded systems, it is necessary to communicate details of the system across several fundamentally different environments (e.g., hardware, the Linux kernel, and application software). Rather than using lots of “magic” numbers stashed in cryptic header files, this is now mostly done at the system level with device-tree. Details provided via device-tree can control which kernel modules get loaded (or do not), as well as modify the behavior of those modules.

A HARDWARE IP

- Vendor IP

  Standard vendor IP blocks for GPIO and I2C are implemented along with the Hard IP ARM cores in the example design files. These vendor IP blocks are added to the device-tree and are controlled by standard Linux kernel drivers. This allows the pushbutton switches and LEDs connected to the FPGA fabric to be made available to the Linux kernel as part of the standard gpio subsystem.

- Xilinx HDMI IP

  The ZCU104 uses the Xilinx HDMI IP core, however the Linux drivers for this core are disabled since the NDI encoding data flow operates outside of the Linux kernel’s standard video input and output systems. The control software for the HDMI core instead operates on one of the Cortex-R5 cores, which means all the hardware used by the HDMI core (uart1, i2c0, i2c1) must be disabled in Linux to avoid contention.

- Altera HDMI IP

  The Altera examples use the Altera HDMI IP cores. Management of these IP cores is done outside of Linux using the NIOS CPU core instantiated as part of the Altera HDMI example design. This code is used as-is for HDMI Rx, and modified to allow configuration of the HDMI Tx logic to support several different resolutions for the NDI Decode example design.

- Custom IP

  The NDI hardware implements a 4K block of register space which is manually added to the device-tree along with IRQ settings and other hardware details. The generic-uio driver is used to map the hardware register space and IRQs so they are available to user-space code.

B RESERVED MEMORY REGIONS

This design requires two large blocks of buffer memory, one for compressed video and raw audio data that must be visible to Linux, and one for raw video data which can be visible to Linux but may also be implemented as a separate memory bank for improved system performance.

- NewTek_Reserved

  This memory region is used to store compressed video data and raw audio data. This region is marked cacheable for performance. Cache coherency is maintained since the hardware accesses this memory region using a cache coherent bus interface.

- NewTek_Video

  This memory region is used to store raw (uncompressed) video data and thus requires high bandwidth. The application does not access this memory in normal operation so this region may be implemented as a dedicated FPGA-side memory bank to improve performance. This region will only appear in the device-tree if implemented using shared memory visible to Linux. If a live video input is not available and this memory region is visible to Linux, a video pattern can be written to allow testing of the compression core and software application.

C SOFTWARE

- Standard Vendor IP Blocks

  The standard Vendor IP blocks (for I2C and GPIO) are added to the device tree and stock Linux kernel drivers are used to communicate with this hardware.

- Custom IP blocks

  The device-tree entries for the custom FPGA IP use the generic-uio kernel driver. This driver makes the memory regions and IRQs specified by the device-tree entries accessible to user-space code. The libuio library is used to facilitate accessing the hardware.

  The application also reads details regarding the reserved memory regions directly from the device-tree, as this functionality is not supported by libuio.

1.4.1.2 INITIAL STARTUP

File(s)	Description
`<device-tree>`	Contains memory addresses and IRQ details
`src/cpp/ndi_common/hardware.*`	Low-level access to register space and IRQs
`src/cpp/ndi_common/device.*`	Machine specific details
`src/fpga/ip/common/Version.vhd`	Version information compiled into the hardware
`src/cpp/ndi_encode/ndi_encode.*`	Top level application file (Encode)
`src/cpp/ndi_decode/ndi_decode.*`	Top level application file (Decode)

When the application is launched, quite a bit of setup is performed before continuing operation is passed to a number of created threads. Most of this setup happens when the hardware class is initialized. After the software processes any command-line options, an instance of the hardware class is created and initialized.

The hardware class uses libuio to access the memory regions and interrupts specified in the device-tree for the custom hardware IP. In addition, details for the reserved memory regions are read from the device-tree and the memory is mmap()'d to make it available for access. At this point, version information is read from the hardware and a device class specific to the hardware platform is created. This allows some behaviors to change between different physical boards (currently used to control any required ACP address translation). Finally, details regarding the UIO devices and reserved memory are printed to the console, and the reserved memory region for compressed video is written with the value 0xdeaddead, making it easier to identify uninitialized memory locations when debugging.

1.4.2 NDI ENCODE OPERATION

1.4.2.1 VIDEO INPUT

File(s)	Description
`src/cpp/ndi_encode/video_capture.*`	`video_capture::` class
`src/cpp/ndi_encode/track.*`	`track::` class
`src/fpga/ip/common/Vid_Track.vhd`	video format tracking hardware
`src/fpga/ip/common/Vid_In.vhd`	Video input DMA logic
`src/fpga/ip/common/Preview.vhd`	Creates low-resolution preview stream
Filename varies	Low level video input logic

Video input starts with the low level video input logic. This logic is platform specific:

Platform	Dir	Implementation
a10socdk	Rx	Altera HDMI RxTx example design
Agilex-7	Rx	Altera HDMI RxTx example design
ZCU104	Rx	Xilinx HDMI Rx example design
Zybo-Z7-20	Rx	Open-source HDMI Rx logic

The software for controlling the low-level video input logic is currently outside the scope of this example. The Zybo-Z7 HDMI input requires no control software, the Xilinx HDMI control software is running bare-metal on one of the Cortex-R5 cores, and the Altera HDMI control software runs on a dedicated NIOS core. See the "HDMI Input Logic" section under "FPGA Projects" for additional details.

There is, however, software support for low-level input monitoring code to send details extracted by the `track::` class (locked/unlocked, resolution, etc) to the `video_capture::` class. See `video_capture::set_signal()` and the `do_commands()` function in `ndi_send.cpp` for details.

WARNING! The provided video input logic is intended as an easy to understand minimally functional example and is not recommended for use in production. In particular, the input logic does not gracefully handle corrupt video data, unexpected cable removal, and similar signal quality issues.

It is expected the user of this SDK will have their own custom video I/O logic specific to their target hardware. If this is not the case and a production capable video I/O solution is desired, there are numerous IP cores available from Xilinx, Altera, and 3rd parties capable of supporting all major video interfaces (SDI, HDMI, Display Port, CSI/DSI, etc.).

Once the low-level video signal is received by the FPGA hardware, the signal is synchronized to the main clock and converted to a standard interface (`HDMI_T`). This signal is sent to the format tracking logic (`Vid_Track.vhd`) and is filtered (`Preview.vhd`) to create a low resolution preview stream. The resulting full and preview resolution streams are each sent to a bus mastering DMA engine (`Vid_In.vhd`) which assembles pixels into words and writes the raw video data into memory.

The `track::` class monitors the video format tracking hardware to detect the acquisition or loss of signal lock. When either event happens, appropriate details are communicated to the `video_capture::` class to start or stop video streaming.

The `video_capture::` class manages the recording of video data into the reserved memory buffer as well as hardware settings controlling things like the data format and preview decimation ratio. Since there are two input streams, the `video_capture::` class operates on frame pairs. Each pair always contains a full resolution frame and may or may not contain a preview resolution frame. The maximum preview framerate is 30 fps, so if the full framerate is greater than 30 fps, some full frames will not have a corresponding preview frame. For simplicity, this is represented as a frame pair with an invalid preview pointer (`NULL`).

The `video_capture::capture_frames` thread loops through the following process to capture frame pairs:

- Create a new frame pair
    - Allocate memory from the reserved buffer
    - Fill in frame details
- Queue the frame (send details to the hardware)
- Post the frame (tell hardware the new frame data is valid)
- Wait for an interrupt indicating the frame has been captured
- Send the frame to the compression logic

Once a frame pair is captured, it is placed on a work queue for the video compression engine. If this queue is too full, the oldest pending frame pair is dropped and a warning is printed.

NOTE: The video input logic assumes a “clean” video signal and is an example design intended to be simple and easy to understand. This logic does not attempt to deal with real world problems like random noise on an unterminated SDI input or when users plug/unplug the cable while the system is running. An actual production device should be more tolerant of signal errors.

1.4.2.2 VIDEO PATTERN

File(s)	Description
`src/cpp/ndi_encode/video_pattern.*`	`video_pattern::` class

The `video_pattern::` class is a sub-class of the `video_capture::` class that does not rely on hardware to generate a video stream. Rather than capturing frames from hardware, this class generates a video pattern any time the signal format changes (it is possible to manually change the resolution at run-time from the console). The video pattern generated is twice as tall as the video format (see the m_scroll_dist member variable), and a preview resolution version of the pattern is also created (since both full and preview resolution streams are required). In normal operation, a frame-sized window of the over-sized video pattern is used to provide source video for the

video_compress:: class, with the start point of the window moving down one line with each new frame, providing the illusion of moving video.

1.4.2.3 VIDEO COMPRESSION

File(s)	Description
src/cpp/ndi_encode/video_compress.*	video_compress:: class
src/fpga/NDI_Enc/Encode_x4.vhd	4-core NDI Hardware Encoder
src/fpga/NDI_Enc/Encode_Xilinx.vhdp	Single NDI Hardware Encoder core (Xilinx)
src/fpga/NDI_Enc/Encode_Altera.vhd	Single NDI Hardware Encoder core (Altera)

One, two, or four copies of the single NDI Encoder core are instantiated in hardware (Encode_x4.vhd). The cores operate in parallel, with each core operating on one or more quarters of the video frame known as a "slice". This increases the effective throughput of the compression core and lowers system latency. The Encode_x4.vhd module also exposes several generics which can be used to configure various aspects of the encoder core.

The video_compress:: class monitors a work queue filled with raw video frames by the video_capture:: class (above). When a new frame pair is received the full resolution frame is sent to hardware for processing. Once the hardware generates an interrupt indicating processing is finished, the software performs minimal post-processing on the compression thread. Slice lengths are read from the hardware and written to memory by fixup_frame(), the resulting frame length is used to update the quality setting, then the frame is passed to the NDI send threads via add_frame_ndi().

The send threads (send_full, send_prvw) monitor independent work queues for the full and preview resolution video streams and pass new frames to the NDI stack. Sending a frame to the NDI stack is a synchronizing event which will block until the frame is fully processed, which is why there are independent threads for the two video streams. This gives each stream the maximum amount of time to send data to the listeners.

1.4.2.4 NDI TRANSMISSION

File(s)	Description
src/cpp/ndi_encode/video_compress.*	video_compress:: class
src/cpp/ndi_encode/network_send_video.cpp	network_send:: video support

The send_full() and send_prvw() threads in the video_compress:: class monitor independent work queues for the full and preview resolution video streams and pass new frames to network_send::add_frame(). This function is basically a shim which converts the video_compress::frame_t type into an NDIlib_video_frame_v2_t type and sends it to the NDI stack. Sending a frame to the NDI stack is a synchronizing event which will block until the frame is fully processed, which is why there are independent threads for the two video streams. This gives each stream the maximum amount of time to send data to the listeners.

NOTE: The various frame types exist primarily because much of the example code predates the availability of the official NDI Advanced SDK. Expect future releases of this code to migrate to using native NDI frame types.

1.4.2.5 AUDIO INPUT

File(s)	Description
src/cpp/ndi_encode/audio_capture.*	audio_capture:: class
src/fpga/ip/common/Aud_In.vhd	Audio input DMA logic

Audio input starts with the low level audio input logic. This logic supports standard I2S serial audio streams. Support for parallel audio data extracted from the HDMI streams will be supported soon. Once assembled into complete audio samples, the data is queued in a FIFO allowing burst writes to system memory.

The audio_capture:: class manages the recording of audio data into the reserved memory buffer as well as hardware settings controlling things like the data format (typically left-justified).

The audio_capture::capture_frames thread loops through a process very similar to the video_capture thread. One major difference is the audio logic currently supports “overlapped” commands, meaning the next command is written to hardware while the current command is still in progress:

- Create a new frame
    - Allocate memory from the reserved buffer
    - Fill in frame details
- Queue the frame (send details to the hardware)
- Post the frame (tell hardware the new frame data is valid)
- Loop until signaled to exit
    - Create a new frame
        - Allocate memory from the reserved buffer
        - Fill in frame details
    - Queue the frame (send details to the hardware)
    - Post the frame (tell hardware the new frame data is valid)
    - Wait for an interrupt indicating the frame has been captured
    - Send the frame to the compression logic

1.4.2.6 AUDIO COMPRESSION

File(s)	Description
src/cpp/ndi_encode/audio_compress.*	audio_compress:: class

This class simply passes captured audio frames from the audio_capture:: class to the NDI stack. It exists primarily to match the video path data flow, to allow any processing of the audio samples that might be required (eg: masking lower-order bits that might contain AES packet data), and because when this code was initially written the NDI API did not support 32-bit signed audio samples.

In future versions, expect the native NDI types to be used and the audio_compress class to be removed.

1.4.2.7 AUDIO TRANSMISSION

File(s)	Description
src/cpp/ndi_encode/network_send_audio.cpp	network_send:: audio support

This function is basically a shim which converts the audio_capture::frame_t type into an NDIlib_audio_frame_interleaved_32s_t type and sends it to the NDI stack.

Expect this code to be deprecated in future versions and the sending logic to be migrated to the audio_capture:: class.

1.4.2.8 TALLY OPERATION

File(s)	Description
src/cpp/ndi_encode/tally.*	tally:: class

Tally outputs are implemented using the Linux kernel LED class. Tally LED entries are created in the device-tree, and the application uses the resulting sysfs entries to control the LED behavior.

The tally class initializes all LEDs to a known state on construction, then the tally process simply loops, checking for updates from the NDI stack. If new tally data is available, the tally LEDs are updated

1.4.3 NDI DECODE OPERATION

1.4.3.1 VIDEO OUTPUT

File(s)	Description
`src/cpp/ndi_decode/video_playback.*`	`video_playback::` class
`src/vpga/ip/common/Test_Pattern_Gen_YUV444`	Video timing & pattern generator
`src/fpga/ip/common/Vid_Out.vhd`	Video output DMA logic
Filename varies	Low level video outputlogic

Video output timing is driven by the low level video logic. This logic is platform specific:

Platform	Dir	Implementation
a10socdk	Tx	Altera HDMI RxTx example design modified for Tx only
ZCU104	Tx	Xilinx HDMI Tx example design
Zybo-Z7-20	Tx	Open-source DVI Tx logic

The software for controlling the low-level video output logic is currently outside the scope of this example. The Zybo-Z7 HDMI output requires no control software, the ZCU104 HDMI control software is running bare-metal on one of the Cortex-R5 cores, and the Altera HDMI control software runs on a dedicated NIOS core. See the "HDMI Output Logic" section under "FPGA Projects" for additional details.

WARNING! The provided video output logic is intended as an easy to understand minimally functional example and is not recommended for use in production. In particular, the output logic mostly does not gracefully handle updating the output format and pixel clock rate. In addition, the timing of receiving, decoding, and queueing frames for output is a complex tradeoff balancing overall latency against consistent real-time performance (never missing the deadline for displaying a newly decoded frame). This example uses the output vertical sync interrupt to drive all output timing which is likely not the ideal implementation for most use cases.

It is expected the user of this SDK will have their own custom video I/O logic specific to their target hardware. If this is not the case and a production capable video I/O solution is desired, there are numerous IP cores available from Xilinx, Altera, and 3rd parties capable of supporting all major video interfaces (SDI, HDMI, Display Port, CSI/DSI, etc.).

The video output logic generates a pixel stream with programmable timings based on the video clock which is programmable on most platforms to support multiple resolutions. The video pixel data can be a hardware generated test pattern or the video data can be filled in with data from memory for video playback. The vertical sync interrupt from `Vid_Out.vhd` is used to drive all output operations, and an NDI FrameSync instance is used to synchronize the received NDI video stream with the hardware output timing.

1.4.3.2 VIDEO DECOMPRESSION

File(s)	Description
`src/cpp/ndi_decode/video_decode.*`	`video_decode::` class
`src/fpga/NDI_Dec/Decode_x4.vhd`	4-core NDI Hardware Encoder
`src/fpga/NDI_Dec/Decode_Xilinx.vhdp`	Single NDI Hardware Encoder core (Xilinx)
`src/fpga/NDI_Dec/Decode_Altera.vhd`	Single NDI Hardware Encoder core (Altera)

One, two, or four copies of the NDI Decoder core are instantiated in hardware (`Decode_x4.vhd`). The cores operate in parallel, with each core operating on one or more quarters of the video frame known as a "slice". This increases the effective throughput of the compression core and lowers system latency. The `Decode_x4.vhd` module also exposes several generics which can be used to configure various aspects of the decoder core.

The `video_decode::` class monitors a work queue filled with NDI video frames by the `video_playback::` class (above). When a new frame is received it is sent to hardware for processing. Once the hardware generates an interrupt indicating processing is finished, the software queues the decoded frame for display via `video_playback::send_frame()`.

1.4.3.3 VIDEO OUTPUT PROCESS FLOW:

- video_playback::pull_frames thread
  - Wait for vertical sync interrupt: `video_playback::pull_frames`
  - Pull a frame from the NDI FrameSync Instance: `network_recv::get_video()`
  - Copy video frame to reserved memory region visible by hardware: `video_decode::add_frame()`
  - Push copied frame onto NDI Decode queue: `video_decode::add_frame()`
  - Free video frame acquired from the NDI FrameSync Instance
- video_decode::decode_frames thread
  - Wait for new frame to decode
  - Setup hardware to decode NDI frame into raw video data: `video_decode::decode_frame()`
  - Start hardware decoding: `video_decode::post_frame()`
  - Wait for interrupt
  - Queue the raw frame for display: `video_playback::send_frame()`
  - The frame will display after the next vertical sync

1.4.3.4 AUDIO OUTPUT

File(s)	Description
src/cpp/ndi_decode/audio_playback.*	`audio_playback::` class
src/fpga/ip/common/Aud_Out.vhd	Audio output DMA logic

Audio output starts with the low level output input logic. This logic supports standard I2S serial audio streams. Support for parallel audio data extracted from the HDMI streams will be supported soon. Once assembled into complete audio samples, the data is queued in a FIFO allowing burst writes to system memory.

The `audio_playback::` class manages the playback of audio data from the reserved memory buffer as well as hardware settings controlling things like the data format (typically left-justified).

The `audio_playback::pull_frames` thread loops through a process very similar to the video_playback thread. An interrupt is generated when a programmed audio DMA completes which drives the timing of the rest of the audio output operations.

1.4.3.5 AUDIO OUTPUT PROCESS FLOW

- audio_playback::pull_frames thread
  - Wait for audio interrupt: `audio_playback::pull_frames`
  - Pull a frame from the NDI FrameSync Instance: `network_recv::get_audio()`
  - Convert audio frame to 32-bit data and copy to reserved memory region visible by hardware: `audio_playback::queue_frame()`
  - Free video frame acquired from the NDI FrameSync Instance
  - Enable playabck of audio data: `audio_playback::post_frame()`

1.4.4 SUPPORTED VIDEO FORMATS

The NDI Encode and Decode FPGA cores support a variety of standard and custom in-memory video formats for 4:2:2 and 4:2:0 YUV video. Planar alpha is also supported for 4:2:2 formats. For further details, see the `generate_video()` routine in `src/cpp/ndi_encode/video_pattern.cpp`.

Support for semi-planar formats and planar alpha can be

1.4.4.1 UYVY

4:2:2 Packed 8-bit

```
Addr : Component
  00 : U0
  01 : Y0
  02 : V0
  03 : Y1
  ...
```

1.4.4.2 YUYV

4:2:2 Packed 8-bit, Y first

```
Addr : Component
  00 : Y0
  01 : U0
  02 : Y1
  03 : V0
  ...
```

1.4.4.3 NV16

4:2:2 Semi-planar 8-bit - luma then packed chroma

```
Plane 0 (Luma)
Addr : Component
  00 : Y0
  01 : Y1
  02 : Y2
  03 : Y3
  ...

Plane 1 (Chroma)
Addr : Component
  00 : U0
  01 : V0
  02 : U1
  03 : V1
  ...
```

1.4.4.4 UYVW

4:2:2 Packed 16-bit (custom format) - 16-bit version of UYVY

```
Addr : Component
  00 : Y0
  02 : U0
  04 : Y1
  06 : V0
  ...
```

1.4.4.5 Y216

4:2:2 Packed 16-bit, Y first - 16-bit version of YUYV

```
Addr : Component
  00 : Y0
  02 : U0
  04 : Y1
  06 : V0
  ...
```

1.4.4.6 P216

4:2:2 Semi-planar 16-bit - luma then packed chroma

```
Plane 0 (Luma)
Addr : Component
  00 : Y0
  02 : Y1
  04 : Y2
  06 : Y3
  ...

Plane 1 (Chroma)
Addr : Component
  00 : U0
  02 : V0
  04 : U1
  06 : V1
  ...
```

1.4.4.7 420

4:2:0 Packed 8-bit (custom format) - Macroblock interleaved, 16 pixels of Y followed by 8 pixels of U (even lines) or V (odd lines)

```
Addr : Component
0x00 : Y0
0x01 : Y1
...
0x0E : Y14
0x0F : Y15
0x10 : U0
0x11 : U1
...
0x16 : U6
0x17 : U7

0x18 : Y16
0x19 : Y17
...
```

1.4.4.8 NV12

4:2:0 Semi-planar 8-bit - Luma then packed chroma

```
Plane 0 (Luma)
Addr : Component
  00 : Y0
  02 : Y1
  04 : Y2
  06 : Y3
  ...

Plane 1 (Chroma)
Addr : Component
  00 : U0
  02 : V0
  04 : U1
  06 : V1
  ...
```

1.4.4.9 420W

4:2:0 Packed 16-bit (custom format) - Macroblock interleaved, 16 pixels of Y followed by 8 pixels of U (even lines) or V (odd lines)

```
Addr : Component
0x00 : Y0
0x02 : Y1
...
0x1C : Y14
0x1E : Y15
0x20 : U0
0x22 : U1
...
0x2C : U6
0x2E : U7

0x30 : Y16
0x32 : Y17
...
```

1.4.4.10 P016

4:2:0 Semi-planar 16-bit - Luma then packed chroma

```
Plane 0 (Luma)
Addr : Component
  00 : Y0
  02 : Y1
  04 : Y2
  06 : Y3
  ...

Plane 1 (Chroma)
Addr : Component
  00 : U0
  02 : V0
  04 : U1
  06 : V1
  ...
```

1.5 REBUILDING FROM SOURCE

The FPGA reference design is broken into four major components, each with it's own subdirectory under the `fpga_reference_design/` directory:

Directory	Description
`src/fpga/`	FPGA Project Files
`linux_kernel/`	Linux kernel, U-Boot, and device-tree
`src/cpp/`	C++ Application Project Files
`os_uSD/`	Debian rootfs and uSD images

1.5.1 FPGA HARDWARE DESIGN

The `fpga_reference_design/src/fpga/` directory contains projects to build a bit-file for each supported platform. Refer to the "FPGA Projects" section for full details on building the hardware projects.

1.5.2 BOOT LOADER, LINUX KERNEL, AND DEVICE-TREE

The `fpga_reference_design/linux_kernel/` directory contains projects to build a boot-loader, kernel, and device-tree for the supported platforms. The device-tree customizations required for the NDI FPGA hardware are also included in the example projects. Refer to the "Linux Kernel and Bootloader" section for full details.

1.5.2.1 PETALINUX ROOT FILE SYSTEM

NOTE:

For Xilinx platforms, the scripts in `fpga_reference_design/linux_kernel` create a Petalinux root file-system in addition to the kernel and boot loader. This is a very minimal embedded system and it is difficult to customize since the entire OS is cross-compiled. This example ignores the Petalinux generated rootfs and instead uses a standard Debian OS install which allows for self-hosted compiles and easy modification of the OS using standard package management tools.

For details on building the Debian root file-system, see the "Bootable uSD image" section (below).

1.5.3 APPLICATION SOFTWARE

The `fpga_reference_design/src/cpp/` directory contains example encode and decode software applications which use the FPGA based NDI logic to (de)compress live video data, and the Advanced NDI SDK to send and receive that data as an NDI stream. Refer to the "C++ Application Code" section for details.

1.5.4 BOOTABLE USD IMAGE

The `fpga_reference_design/os_uSD/` directory contains scripts to create a Debian root filesystem as well as a bootable uSD card image. The resulting images contain all prerequisites needed to perform a self-hosted build of the example NDI application (ie: built on the ARM platform and not cross-compiled). Refer to the "uSD Image Builder" section for full details.

2 FPGA PROJECTS

2.1 DIRECTORY STRUCTURE

Source code for the FPGA example projects is organized into the following subdirectories under the `fpga_reference_design/src/fpga/` directory:

Directory	Contents
NDI_Dec/	VHDL files for the NDI Decoder core. All files should be compiled into the NDI_Dec VHDL library rather than the default work library
NDI_Enc/	VHDL files for the NDI Encoder core. All files should be compiled into the NDI_Enc VHDL library rather than the default work library
ip/altera	Altera specific implementations of memory and FIFO blocks
ip/common	Source code common to all example designs
ip/extern	External (3rd party) IP used in the example designs
ip/packages	VHDL Packages defining various types used in the logic
ip/xilinx	Xilinx specific implementations of memory and FIFO blocks
Agilex7-Enc	Quartus Pro 25.1.1 NDI Encode project directory targeting the DK-SI-AGI027FA Agilex-7 SoC DevKit
a10-socdk-enc/	Quartus Pro 22.2 NDI Encode project directory targeting the Arria-10 SoC DevKit
a10-socdk-dec/	Quartus Pro 22.2 NDI Decode project directory targeting the Arria-10 SoC DevKit
SoCKit-Dec/	**Deprecated** Quartus 22.1 NDI Decode project directory targeting the Terasic SoCKit
ZCU104-Enc/	Vivado 2022.1 NDI Encode project directory targeting the Xilinx ZCU104
ZCU104-Dec/	Vivado 2022.1 NDI Decode project directory targeting the Xilinx ZCU104
Zybo-Z7-20-Enc/	Vivado 2024.2 NDI Encode project directory targeting the Digilent Zybo-Z7-20
Zybo-Z7-20-Dec/	Vivado 2024.2 NDI Decode project directory targeting the Digilent Zybo-Z7-20
Zybo-Z7-20-Enc-Lite/	"Lightweight" Vivado 2024.2 NDI Encode project directory targeting the Digilent Zybo-Z7-20 with 16-bit SDRAM interface
Arty-Z7-20-Enc/	Vivado 2024.2 NDI Encode project directory targeting the Digilent Arty-Z7-20
KV260-Enc/	Vivado 2022.1 NDI Encode project directory targeting the Xilinx Kria KV260

2.2 BUILD DEPENDENCIES

2.2.1 XILINX

2.2.1.1 ZCU104

These projects require Vivado version 2022.1. Any edition of Vivado 2022.1 will work, including the "WebPACK" edition available via free download from Xilinx.

You must have a valid license for the Xilinx HDMI IP core used in the ZCU104-based example designs. You may use a full license or a free evaluation license, but the license **must** be installed **prior to launching Vivado** to build the project: https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/hdmi.html

2.2.1.2 ZYBO-Z7-20 / ZYBO-Z7-20-LITE / ARTY-Z7-20

These projects require Vivado version 2024.2. Any edition of Vivado 2024.2 will work, including the "WebPACK" edition available via free download from Xilinx.

Board files from Digilent **must** be installed **prior to launching Vivado** in order to build the Zybo example designs. The board files may be obtained from github and installation instructions are available on the Digilent website.

Digilent periodically updates their board files repository which can lead to errors when opening projects caused by a discrepancy in the `BoardPart` property. For reference, the current version of Zybo and Arty example designs were built with commit hash `3f67d8f827bf83bb6a16b4879e4fdad68b8443e8`. The ZCU104 example designs are built with board file revision 1.1 and should have been installed by Vivado 2022.1. This can be verified from the Vivado Tcl console with the command `get_boards *zcu104*`. Older or missing board files are a common source of failures to build Xilinx example designs.

Due to changes in Xilinx recommendations, these projects show critical warnings when loaded in recent versions of Vivado. These warnings may be ignored, as explained by the Digilent reference manual.

2.2.1.3 KRIA KV260

These projects require Vivado version 2022.1. Any edition of Vivado 2022.1 will work, including the "WebPACK" edition available via free download from Xilinx.

The Kria KV260 example design deviates significantly from others supplied with the NDI Advanced SDK. See the `README` file in the `src/fpga/KV260-Enc` directory for details on how to create designs targeting this platform.

2.2.2 ALTERA

The NDI IP cores are available in two versions for Altera. Other than the encryption method used, these two versions are identical.

Designs using newer versions of Quartus (currently the Arria-10 and Agilex-7 examples) should use the `*_Altera_1735.vhd` files which are encrypted using IEEE 1735 and support development with Quartus as well as simulation with Synopsis and Mentor products.

For legacy Quartus versions which require a license file, the `*_Altera.vhd` files should be used and the contents of the file `NDI_Enc/Encode_Altera.lic` (for Encode) and `NDI_Dec/Decode_Altera.lic` (for Decode) must be appended to your Quartus license file. If you do not have a license file, you must create one.

Agilex-7 projects require Quartus Pro 25.1.1.

Arria-10 projects require Quartus Pro 22.2.0.

Cyclone-V projects require Quartus 22.1. The examples were compiled using the free "Lite" version, however the "Standard" edition should work as well.

Follow the Altera instructions for installation, including the required manual installation of WSL if you are using Windows.

If you plan to rebuild the NIOS code which reconfigures the GXB transceivers and PLLs, you need to manually install Eclipse per the Altera Quartus installation instructions. These instructions can also typically be found in the file:
`<Quartus installation directory>/nios2eds/bin/README`

2.3 REBUILDING

2.3.1 XILINX

All required dependencies (see above) **MUST** be installed **prior to launching Vivado**. Once you have all required dependencies installed, simply open the desired project and build normally. The contents for the block design elements (hard processor system, PLLs, etc) will be regenerated on the first build.

2.3.2 ALTERA

2.3.2.1 AGILEX-7 PROJECTS

To rebuild the Agilex-7 projects, simply open the project and Start Compilation. When opening the project for the first time, you will see the following warning:

```
Warning(125092): Tcl Script file support_logic/agx_hdmi21_frl_demo_auto_tiles.qip not found.
```

This warning can be safely ignored as the missing file is created during the Support-Logic Generation step of the build process.

After the build is finished, the rbf and jic files must be created. See the `mk.jic.sh` file in the `quartus/output_files` directory of the corresponding project for example commands, which must be run from a NIOS V command shell.

2.3.2.2 OTHER ALTERA PROJECTS

For other Altera projects, you must generate the hps platform logic from the qsys file as described below.

A REBUILD THE HPS PLATFORM

- Launch Platform Designer
- Open ndi_hps.qsys
- Click “Generate HDL…”
- Select VHDL Synthesis output (optional)
- Click “Generate”
- Once the qsys project has been generated, run a normal Quartus compilation

2.4 KNOWN ISSUES AND LIMITATIONS:

The video I/O logic (eg: HDMI to SDRAM and SDRAM to HDMI) is intended as an easy to understand minimally functional example and is not recommended for use in production. In particular, the input logic does not gracefully handle corrupt video data, unexpected cable removal, and similar signal quality issues.

It is expected the user of this SDK will have their own custom video I/O logic specific to their target hardware. If this is not the case and a production capable video I/O solution is desired, there are numerous IP cores available from Xilinx, Altera, and 3rd parties capable of supporting all major video interfaces (SDI, HDMI, Display Port, CSI/DSI, etc.).

2.5 NDI IP FILES AND USE

2.5.1 IP CORE CONFIGURATION

The NDI encoder and decoder cores expose three generics that allow users to enable or disable some features at compile time. This can reduce the amount of hardware resources for each instantiated core. In particular, disabling 16-bit formats by setting `FORMAT_16B => false` will significantly reduce the block RAM utilization of the raw video buffers.

`PLANAR_ALPHA : boolean`

- Enables support for a separate planar alpha channel

`PLANAR_VIDEO : boolean`

- Enables support for semi-planar video formats (NV16, P216, NV12, P016)

`FORMAT_16B : boolean`

- Enables support for 16-bit formats (UYVW, Y216, P216, 420W, P016)

2.5.2 NDI ENCODER

Two VHDL files are required to use the NDI Encoder. Both files should be compiled into the `NDI_Enc` library in the following order:

- Xilinx Projects:

```
`Encode_Xilinx.vhdp`
`Encode_x4.vhd`
```

- Legacy Altera Projects:

```
`Encode_Altera.vhd`
`Encode_x4.vhd`
```

- Recent Altera Projects:

```
`Encode_Altera_1735.vhd`
`Encode_x4.vhd`
```

There is also a component declaration file `Enc_Core_Comp.vhd` for use as a reference when instantiating the encoder core. Each encoder core `Enc_Core_E` includes a register I/O interface, a read-only bus master interface for reading raw video data, and a write-only bus master interface for writing compressed NDI data. Note the raw video memory and the compressed NDI memory do not need to be the same physical bank of memory. Using two memory banks can improve system performance, particularly on lower-end devices such as the Zynq 7000 and Cyclone-V families.

2.5.3 NDI DECODER

Two VHDL files are required to use the NDI Decoder. Both files should be compiled into the `NDI_Dec` library in the following order:

- Xilinx Projects:

```
`Decode_Xilinx.vhdp`
`Decode_x4.vhd`
```

- Legacy Altera Projects:

```
`Decode_Altera.vhd`
`Decode_x4.vhd`
```

- Recent Altera Projects:

```
`Decode_Altera_1735.vhd`
`Decode_x4.vhd`
```

There is also a component declaration file `Dec_Core_Comp.vhd` for use as a reference when instantiating the encoder core. Each decoder core `Dec_Core_E` includes a register I/O interface, a read-only bus master interface for reading compressed NDI data, and a write-only bus master interface for writing raw video data. Note the raw video memory and the compressed NDI memory do not need to be the same physical bank of memory. Using two memory banks can improve system performance, particularly on lower-end devices such as the Zynq 7000 and Cyclone-V families.

2.5.4 SDRAM BUS INTERFACES

The bus-mastering memory interfaces are natively a sub-set of the Altera Avalon interface specification. These buses may be tied directly to an Avalon interface port in an Altera design. For Xilinx designs, clear-text bus shim logic is provided to convert the Avalon bus subset used by the Encoder core into a more standard AXI interface:

File	Function
Avl_Axi_Rd.vhd	Translates Avalon read bus to AXI read bus
Avl_Axi_Wr.vhd	Translates Avalon write bus to AXI write bus
Avl2_Axi_rd.vhd	Merges 2 Avalon read buses into one AXI read bus
AXI_Reg_Wr.vhd	Interface AXI write bus to simple register write bus
AXI_Reg_Rd.vhd	Interface AXI read bus to simple register read bus

Usage examples for these files can be found in the top-level files for the example designs.

2.5.4.1 NDI ENCODE

A WRITES (COMPRESSED NDI DATA)

There is clear-text logic in the Encode_x4 file which merges the four lower bandwidth compressed write streams into a single write stream. The raw (uncompressed) audio data is also merged into this stream.

This write bus is connected to a cache coherent port to the hard processor system, eliminating the need for a kernel mode DMA driver.

B READS (RAW VIDEO DATA)

For maximum performance, the four read streams are all brought out to the top level to provide maximum flexibility in scheduling SDRAM transactions. The NDI Encoder cores will tolerate fairly high latency on raw video reads, but require sufficient overall bandwidth to avoid stalling the NDI cores. Each of the NDI cores can process one pixel element (Y or C) per clock.

2.5.4.2 NDI DECODE

A READS (COMPRESSED NDI DATA)

There is clear-text logic in the Decode_x4 file which merges the four lower bandwidth compressed Read streams into a single AXI read stream.

This read bus is connected to a cache coherent port to the hard processor system, eliminating the need for a kernel mode DMA driver.

B WRITES (RAW VIDEO DATA)

For maximum performance, the four write streams are all brought out to the top level to provide maximum flexibility in scheduling SDRAM transactions. The NDI Decoder cores will tolerate fairly high latency on raw video writess, but require sufficient overall bandwidth to avoid stalling the NDI cores. Each of the NDI cores can process one pixel element (Y or C) per clock. For maximum performance, the four write streams are all brought out to the top level.

2.5.5 FULL VS. LITE ENCODE DESIGNS

The `Zybo-Z7-20-Lite` and `Arty-Z7-20` example designs are intended as a reference for a more limited platform. Only one SDRAM chip is used (16-bit SDRAM interface) and only two encoder cores are instantiated. This implementation is still capable of operation at resolutions up to 1080p60.

All other example designs instantiate a “full” implementation which includes four NDI Encode or Decode cores providing maximum performance and minimum latency.

2.6 HDMI INPUT LOGIC

2.6.1 AGILEX-7 SOC DEVKIT

The `Agilex7-Enc` design targeting the DK-SI-AGI027FA Agilex-7 SoC Development Kit uses a modified version of the Altera HDMI FPGA IP Design Example. An HPS configuration based on the GHRD example and the NDI logic are added to the top-level HDMI example design file (`agx_hdmi21_frl_demo.v`). Due to the extensive and complex constraints required for the HDMI example design, the top level design file and all HDMI related entity names are unchanged, the rx_vid and rx_audio buses are simply routed to the NDI subsystem as well as the existing rxtx_link module from the HDMI example design.

2.6.2 ARRIA-10 SOC DEVKIT

The `a10-socdk-enc` design targeting the Arria-10 SoC Development Kit uses a slightly modified version of the Altera Arria10 HDMI Rx-Tx Retransmit HDMI example design. The top-level of this example design is modified to export the recevied HDMI video, audio, and AVI data to the top-level where they are connected to the video and audio input logic.

2.6.2.1 REBUILDING THE NIOS HDMI SOFTWARE

- Insure you have manually installed Eclipse per the Altera documentation
- In Quartus, select Tools -> Nios II Software Build Tools for Eclipse
- Use `<NDI SDK Path>/ndi-fpga-refernce/a10-socdk-enc/software` as the Workspace directory
- Create a new project and BSP in Eclipse
    - File -> New -> Nios II Application and BSP from Template
    - SOPC Information File name: `<NDI SDK Path>/ndi-fpga-reference/a10-socdk-enc/rtl/nios/nios.sopcinfo`
    - Project Name: tx_control
    - Project location: `<NDI SDK Path>/ndi-fpga-refernce/a10-socdk-enc/software/tx_control`
        - NOTE: Remove `rtl` from the default path
    - Project Template: Blank Project
    - Click: Finish
- Modify BSP settings
    - Right-click `tx_control_bsp`
    - Nios II -> BSP Editor…
    - In the Main tab, under Settings -> Common
        - Check: enable_small_c_library
        - Check: enable_reduced_device_drivers
        - File -> Save
        - Click: Generate BSP
    - Close BSP Editor
- Using a file browser, navigate to `<NDI SDK Path>/ndi-fpga-refernce/a10-socdk-enc/software/tx_control_src`
- Select all files, right-click, and select: Copy
- In Eclipse, right-click tx_control and select: Paste
- Build: Project -> Build All (or press ctrl+B)
- Update MIF file used to build FPGA
    - Right-click tx_control -> Make Targets -> Build
    - Select mem_init_generate
    - Click: Build

2.6.3 ZYNQ 7000 HDMI INPUT

The Zynq-7000 based designs (i.e., the Arty and Zybo Z7 boards) use RTL based on two open-source projects to process serial HDMI data into the parallel video required by the NDI encoder. Low-level, serial-to-parallel data conversion and lane alignment are performed by RTL from the Digilent DVI2RGB core. The aligned 10-bit characters are then processed as HDMI packets and subpackets by RTL from the Hamsterworks HDMI core. Additional RTL is also provided which merges these two cores together and produces the required video and audio data formats for the NDI encoder. For additional details, see the `src/fpga/ip/extern/README.md` file.

2.6.4 ZCU104 HDMI INPUT

The ZCU104 board uses the Xilinx HDMI 2.0 IP core. A full or evaluation license must be installed for this core prior to building the design with Vivado. This IP core also requires control software to monitor the incoming HDMI signal and make any adjustments required. This software is currently running “bare-metal” on one of the Cortex-R5 cores.

2.6.4.1 REBUILDING THE CORTEX-R5 HDMI SOFTWARE (RX)

- If you have made any changes to the FPGA project:
    - Compile the ZCU104 FPGA design
    - Export the hardware to the SDK
        - Select `File -> Export -> Export Hardware...`
        - Select `<Include bitstream>` and click `Next`
        - Adjust path/filename if desired and click `Finish`
- Launch Vitis Classic IDE
    - This will raise a deprecation warning which can be neglected. The Vitis Unified IDE is not currently recommended.
    - Set the workspace directory as desired
- Create a new platform project `File -> New -> Platform Project`
    - Set platform project name to `ZCU104-Enc`
    - Click `Next`
    - Select `Create new platform from hardware (XSA)`
    - Click `Browse`
    - Select file `ZCU104-Enc.xsa` that was exported from Vivado
    - Click `Next`
    - Set operating system to `standalone`
    - Set CPU to `psu_cortexr5_0`
    - Uncheck `Generate boot components`
    - Click `Finish`
    - Wait for the platform project to be created
- Modify the board support package settings
    - Select `Board Support Package` in the platform editor window
    - Click `Modify BSP Settings`
    - Switch the console to UART1
        - Set the value for stdin to `psu_uart_1`
        - Set the value for stdout to `psu_uart_1`
    - Wait for the BSP regeneration to finish, which can take a while
- Import the HDMI RX example project
    - Select `Board Support Package`
    - Select the `Drivers` tab
    - Select the `v_hdmi_rx_ss` driver
    - Click `Import Examples`
    - Select `RxOnly_R5`
    - Click `OK`
    - Expand `RxOnly_R5_1_system` -> `RxOnly_R5_1` -> `src` (all filenames are relative to this path)
- Update the DDR memory region in the linker script
    - Open the file `src/lscript.ld`
    - Set the `psu_r5_ddr_0_MEM_0` region details to match the rproc_0_reserved reserved memory region in the device-tree and save the file Base Address : 0x3ed00000 Size : 0x80000
- Change HDMI code to use UART1
    - Open `src/xhdmi_example.h`
    - At line 86, change `#define UART_BASEADDR XPAR_XUARTPS_0_BASEADDR` to `#define UART_BASEADDR XPAR_XUARTPS_1_BASEADDR` to use UART1
    - Save the source file

- Update EDID data (optional)
    - Open `src/xhdmi_edid.h`
    - Edit constant `Edid[]` as desired
    - NOTE: EDID contents used for the example are in the file `EDID.NDI.1080p60.dat`
- Build the project `Project -> Build Project`
- Copy the ELF file to the ZCU104
    - The ELF file can be found at the following path
    - `<workspace>/RxOnly_R5_1/Debug/RxOnly_R5_1.elf`
    - Copy this file to `/lib/firmware/` on the ZCU104
    - Rename the file as desired
    - Make sure you reference this filename when launching the R5 via remoteproc!

2.7 HDMI OUTPUT LOGIC

2.7.1 ARRIA-10 SOC DEVKIT

The a10-socdk-dec design targeting the Arria-10 SoC Development Kit uses a modified version of the Altera Arria10 HDMI Rx-Tx Retransmit HDMI example design. All Rx logic and the RxTx loopback logic is removed and the HDMI Tx fpll and iopll blocks use the REFCLK_SDI signal as a clock reference instead of the HDMI Rx clock. The tx_control NIOS code has been updated to reconfigure the output divisor for the fpll and iopll instance to switch between video modes.

2.7.1.1 REBUILDING THE NIOS HDMI SOFTWARE

- Insure you have manually installed Eclipse per the Altera documentation
- In Quartus, select Tools -> Nios II Software Build Tools for Eclipse
- Use `<NDI SDK Path>/ndi-fpga-refernce/a10-socdk-dec/software` as the Workspace directory
- Create a new project and BSP in Eclipse
    - File -> New -> Nios II Application and BSP from Template
    - SOPC Information File name: `<NDI SDK Path>/ndi-fpga-reference/a10-socdk-dec/rtl/nios/nios.sopcinfo`
    - Project Name: tx_control
    - Project location: `<NDI SDK Path>/ndi-fpga-refernce/a10-socdk-dec/software/tx_control`
        - NOTE: Remove `rtl` from the default path
    - Project Template: Blank Project
    - Click: Finish
- Modify BSP settings
    - Right-click `tx_control_bsp`
    - Nios II -> BSP Editor…
    - In the Main tab, under Settings -> Common
        - Check: enable_small_c_library
        - Check: enable_reduced_device_drivers
        - File -> Save
        - Click: Generate BSP
    - Close BSP Editor
- Using a file browser, navigate to `<NDI SDK Path>/ndi-fpga-refernce/a10-socdk-dec/software/tx_control_src`
- Select all files, right-click, and select: Copy
- In Eclipse, right-click tx_control and select: Paste
- Build: Project -> Build All (or press ctrl+B)

- Update MIF file used to build FPGA
    - Right-click tx_control -> Make Targets -> Build
    - Select mem_init_generate
    - Click: Build

2.7.2 ZYBO-Z7-20 HDMI OUTPUT

The Zybo-Z7 board uses the DVI output logic from the "Hamsterworks" HDMI project (ip/extern/HDMI/src/dvid_output.vhd). This is a simple DVI output and no HDMI features (e.g., emedded audio) are currently supported. Both 74.25 MHz (720p) and 148.5 MHz (1080p60) pixel rates are supported.

2.7.3 ZCU104 HDMI OUTPUT

The ZCU104 board uses the Xilinx HDMI 2.0 IP core. A full or evaluation license must be installed for this core prior to building the design with Vivado. This IP core also requires control software to configure the HDMI IP core. This software is currently running "bare-metal" on one of the Cortex-R5 cores.

2.7.3.1 REBUILDING THE CORTEX-R5 HDMI SOFTWARE (TX)

- If you have made any changes to the FPGA project:
    - Compile the ZCU104 FPGA design
    - Export the hardware to the SDK
        - Select `File -> Export -> Export Hardware...`
        - Select `<Include bitstream>` and click `Next`
        - Adjust path/filename if desired and click `Finish`
- Launch Vitis Classic IDE
    - This will raise a deprecation warning which can be neglected. The Vitis Unified IDE is not currently recommended.
    - Set the workspace directory as desired
- Create a new platform project `File -> New -> Platform Project`
    - Set platform project name to `ZCU104-Dec`
    - Click `Next`
    - Select `Create new platform from hardware (XSA)`
    - Click `Browse`
    - Select file `ZCU104-Dec.xsa` that was exported from Vivado
    - Click `Next`
    - Set operating system to `standalone`
    - Set CPU to `psu_cortexr5_0`
    - Uncheck `Generate boot components`
    - Click `Finish`
    - Wait for the platform project to be created
- Modify the board support package settings
    - Select `Board Support Package` in the platform editor window
    - Click `Modify BSP Settings`
    - Switch the console to UART1
        - Set the value for stdin to `psu_uart_1`
        - Set the value for stdout to `psu_uart_1`
    - Wait for the BSP regeneration to finish, which can take a while

- Import the HDMI TX example project
    - Select `Board Support Package`
    - Select the `Drivers` tab
    - Select the `v_hdmi_tx_ss` driver
    - Click `Import Examples`
    - Select `TxOnly_R5`
    - Click `OK`
    - Expand `TxOnly_R5_1_system` -> `TxOnly_R5_1` -> `src` (all filenames are relative to this path)
- Update the DDR memory region in the linker script
    - Open the file `src/lscript.ld`
    - Set the `psu_r5_ddr_0_MEM_0` region details to match the `rproc_0_reserved` reserved memory region in the device-tree and save the file Base Address : 0x3ed00000 Size : 0x80000
    - Save the linker script
- Change the source code to use UART1
    - Open the file `src/xhdmi_example.h`
    - At line 99, change `#define UART_BASEADDR XPAR_XUARTPS_0_BASEADDR` to `#define UART_BASEADDR XPAR_XUARTPS_1_BASEADDR` to use UART1
    - Save the source file
- Update the default video mode
    - Open the file `src/xhdmi_example.c`
    - Edit the default video resolution at line 2506 (optional)
    - Change the colorspace on line 2507 to `XVIDC_CSF_YCRCB_420` for 4Kp60 and `XVIDC_CSF_YCRCB_422` for other modes
    - Possible values are listed in the `XVidC_VideoMode` enum in the file `<workspace>/ZCU104-Dec/psu_cortexr5_0/standalone_domain/bsp/psu_cortexr5_0/include/xvidc.h`
- Fix some build errors due to a missing pattern generator in the hardware design:
    - Copy the tpg header files `xv_tpg.h` and `xv_tpg_hw.h` from the SDK install directory into the `src` directory
        - From: `/tools/Xilinx/Vitis/2024.2/data/embeddedsw/XilinxProcessorIPLib/drivers/v_tpg_v8_6/src/*.h`
        - To: `workspace/TxOnly_R5_1/src`
    - Edit `xhdmi_example.c`
        - Comment the contents of the `XV_ConfigTpg` function (lines 329-380)
        - Comment the contents of the `ResetTpg` function (lines 385-390)
        - Comment the assignment to `Pattern` (line 1360)
        - Comment the call to `XV_ConfigTpg` (line 1363)
        - Save the file
    - Edit `xhdmi_menu.c`
        - Comment the assignment to `Pattern` (line 1514)
        - Comment the call to `XV_ConfigTpg` (line 1517)
        - Save the file
- Build the project `Project` -> `Build Project`
- Copy the ELF to the ZCU104
    - The ELF file can be found at the following path
    - `<workspace>/TxOnly_R5_1/Debug/TxOnly_R5_1.elf`
    - Copy this file to `/lib/firmware/` on the ZCU104
    - Rename the file as desired
    - Make sure you reference this filename when launching the R5 via remoteproc!

2.7.4 SOCKIT-DEC

WARNING: The SoCKit project should be considered deprecated as the development board is no longer available for purchase. This design will be removed from future versions of the NDI Advanced SDK.

The SoCKit-Dec design targeting the Cyclone-V SoCKit Development board video output implementation (DVI_TX.vhd) converts the YUV video data from the NDI decoder into RGB and drives both the on-board VGA output and the (optional) Terasic DVH-HSMC daughtercard via the HSMC connector.

This is a simple DVI output and no HDMI features (e.g., emedded audio) are currently supported. Both 74.25 MHz (720p) and 148.5 MHz (1080p60) pixel rates are supported.

2.8 MIPI CSI INPUT LOGIC

2.8.1 XILINX KRIA KV260

This project uses the Xilinx MIPI CSI-2 Rx subsystem as configured for the “Smart Camera Accelerated Application” from Xilinx. A tap is added to the video output of the MIPI CSI core allowing pixels to be sent to the NDI Encode logic.

3 C++ APPLICATION CODE

A set of example applications are provided which illustrate the use of the FPGA based NDI IP cores with the Advanced NDI SDK. For full details and a theory of operation covering the entire system (software, hardware, and OS), refer to “Theory of Operation” in the “NFI FPGA Reference Design” section.

3.1 DIRECTORY STRUCTURE

Source code for the C++ Application examples is organized into the following subdirectories under the `fpga_reference_design/src/cpp/` directory:

Directory	Contents
ndi_common/	Files common to all NDI applications
ndi_decode/	Example NDI Decoder application (eg: TV or Monitor)
ndi_encode/	Example NDI Encoder application (eg: Camera or NDI Source)
ndi_util/	Debug and utility applications

3.2 GETTING STARTED

3.2.1 PREREQUISITES

The following prerequisites must be installed before you can build the example applications.

3.2.1.1 LIBUIO

Source Code:

- https://github.com/Linutronix/libuio

libuio is used to access the FPGA hardware via entries in the device-tree

The available uSD card images have libuio packages compiled from source (see `~/libuio`) and the debian packages installed. For other platforms, libuio will need to be compiled and installed manually.

3.2.1.2 NDI ADVANCED SDK LIBRARY

The NDI Advanced SDK allows the sending and receiving of compressed frames, which is required to use the FPGA encoder. To install the library, simply copy the files to an appropriate location for your system. The example uSD card images have the NDI library files already installed in `/usr/local/lib` and `/usr/local/include`.

3.2.2 BUILDING AND INSTALLING

Once the prerequisites have been installed, building the applications is straight-forward:

```
cd <NDI_SDK>/fpga_reference_design/src/cpp
make
sudo make install
```

The applications will be installed with setuid privileges to the `/usr/local/bin/` directory.

3.3 NDI_ENCODE

The `ndi_encode` utility is an example application illustrating the use of the FPGA based NDI encoder IP and the Advanced NDI SDK to create an NDI sender.

3.3.1 USAGE

The ndi_encode program is simply launched from the command line, eg:

```
ndi_encode
```

Note that this will launch the version installed in /usr/local/bin/. Alternatively, the program can be run from the build directory. Note that root permissions are required so it is necessary to use sudo:

```
sudo /path/to/workdir/ndi_encode
```

3.3.2 RUN TIME COMMANDS

Once running, the program will listen for commands on stdin. These commands are intended to modify the running behavior of the program and illustrate how to integrate with a separate utility monitoring the physical video input source (eg: an HDMI or SDI input) as a signal is acquired, locked, or lost.

The commands which change the raw video parameters (resolution, frame rate, format, alpha) only apply when running in pattern generator mode. With a real video input, format details are determined automatically by the tracking logic.

- start | stop

  Start or stop streaming video and audio. This would correspond with the acquisition (start) or loss (stop) of a stable signal.

- 720 | 1080 | 4k

  Change the video resolution

- 24 | 25 | 30 | 50 | 60

  Set the primary video frame rate

- 1000 | 1001

  Set the secondary frame rate: 1000=exact (eg: 60.00), 1001=NTSC (eg: 59.94)

- UYVY | YUYV | NV16 | UYVW | Y216 | P216 | 420 | NV12 | 420W | P016

  Set the raw video format

- Alpha+ | Alpha-

  Enable / Disable alpha (4:2:2 modes only)

- exit

  Quit the program

3.3.3 COMMAND LINE OPTIONS

Various command line switches are available to control the program's behavior:

3.3.3.1 GENERAL OPTIONS

- Set default video resolution
    - -1 1080p60 (default)
    - -7 720p60
    - -4 4Kp60
- Video options
    - -P Pattern generator

      Use the built-in pattern generator instead of live video input.
    - -v Increase verbosity

      Output more detailed information. This switch may be provided multiple times to further increase the debugging output. The available levels are:

        - Fatal
        - Error (default)
        - Warning
        - Information
        - Debugging
    - -q Decrease verbosity (quiet)

      Decrease the amount of output generated. This switch may be provided multiple times. See the discussion of -v (above) for details.
- Audio Options
    - -h HDMI input

      Use embedded audio from the HDMI stream.
    - -l Line input

      Use analog audio input.
    - -p Pattern generator

      The audio input hardware includes a simple saw-tooth pattern generator that can be used as an input source for testing.

3.3.3.2 DEBUGGING AND ADVANCED OPTIONS

- -X Disable video capture

  This disables the video capture thread. No video frames will be captured, compressed, or sent to the NDI stack. Only audio will be processed.

- -x Disable audio capture

  This disables the video capture thread. No audio samples will be captured, compressed, or sent to the NDI stack. Only video will be processed.

- -F Full resolution capture only

  This option disables the preview stream and only sends full resolution frames to the NDI stack.

- -f fflush() after each log message

  This is useful if the application is crashing to insure log error messages actually reach the terminal.

- -A Use ACP

- -B Bypass ACP

  These options do nothing on the current Zynq development boards and only apply to Cyclone-V based platforms.

3.4 NDI_DECODE

The `ndi_decode` utility is an example application illustrating the use of the FPGA based NDI encoder IP and the Advanced NDI SDK to create an NDI receiver.

3.4.1 USAGE

The ndi_decode program is simply launched from the command line, eg:

```
# Launch and display the first NDI source found
ndi_decode

# Launch and connect to a specific NDI source
ndi_decode -s "Machine name (NDI channel)"
```

Note that this will launch the version installed in /usr/local/bin/. Alternatively, the program can be run from the build directory. Note that root permissions are required so it is necessary to use sudo:

```
sudo /path/to/workdir/ndi_receive
```

3.4.2 COMMAND LINE OPTIONS

Various command line switches are available to control the program's behavior:

3.4.2.1 GENERAL OPTIONS

- -s `<NDI Source>`

  Specify NDI source to stream. If this setting is not provided, ndi_receive will attempt to connect to the first NDI source it finds on the network.

- Set output video resolution
  - -1 1080p (default)
  - -7 720p
  - -4 4Kp
- Set output video framerate
  - -6 60 Hz (default)
  - -5 50 Hz
- Select output video
  - -k Output key (alpha) data if available instead of image data
- Select SDRAM burst mode
  - -b Disable SDRAM burst mode (e.g., 1 macroblock to SDRAM per burst transfer). Default behavior is to burst 4 macroblocks to SDRAM per burst transfer.
- -o `<output format>`

  Specifies the output video format, where `output format` is one of the enumerated `video_format_t` values in `hardware.h` in decimal. The default is packed 4:2:2 (e.g., UYVY).

3.4.2.2 DEBUGGING AND ADVANCED OPTIONS

- -v Increase verbosity

  Output more detailed information. This switch may be provided multiple times to further increase the debugging output. The available levels are:

  - Fatal
  - Error (default)
  - Warning
  - Information
  - Debugging

- -q Decrease verbosity (quiet)

  Decrease the amount of output generated. This switch may be provided multiple times. See the discussion of -v (above) for details.

- -f fflush() after each log message

  This is useful if the application is crashing to insure log error messages actually reach the terminal.

- -A Use ACP

- -B Bypass ACP

  These options do nothing on the current Zynq development boards and only apply to Cyclone-V based platforms.

- -c `<capture file>`

Allows for individual frames to be captured to the filesystem. This switch is intended for debugging purposes and assumes that the stride is equal to the length of a row of image data. The default is to capture frame number 25, but this can be modified using the `-n` argument to select a different value. No more than one frame can be captured at a time.

- -n `<frame capture number>`

If capturing video frames, indicates the frame number to capture (defaults to 25). Requires the use of the `-c` option to capture frames.

3.5 NDI_REG

The `ndi_reg` utility allows reading and writing of the NDI hardware registers. To read an NDI register, provide the register address on the command line:

```
# Read the Video Input register
ndi_reg 0x200
```

To write an NDI register, provide the register address and write data on the command line:

```
# Write the Video Input register
ndi_reg 0x200 0x00c00100
```

3.5.1 REGISTER MAP

Each major hardware block is allocated a range of control register addresses as detailed below and implemented in the `device.h` and `hardware.h` files in the `src/cpp/ndi_common/` directory.

Hardware	Base Address	Length
NDI Encoder Core 0	0x000	0x40
NDI Encoder Core 1	0x040	0x40
NDI Encoder Core 2	0x080	0x40
NDI Encoder Core 3	0x0c0	0x40
NDI Decoder Core 0	0x100	0x40
NDI Decoder Core 1	0x140	0x40
NDI Decoder Core 2	0x180	0x40
NDI Decoder Core 3	0x1c0	0x40
Video Input	0x200	0x10
Preview Input	0x210	0x10
Audio Input	0x220	0x10
Video Tracking	0x230	0x10
Video Output	0x280	0x10
Audio Output	0x290	0x10
Version (Date)	0x3fe	0x10
Version (Revision)	0x3ff	0x10

For low-level details on register settings and bit definitions refer to the specific C++ class code and headers for the hardware as described in the “Theory of Operation” section.

4 LINUX KERNEL AND BOOT LOADER

To the extent possible, the NDI examples attempt to follow the vendor specific boot flow and methods for building a Linux kernel and U-Boot boot-loader. The following sections detail specific customizations made to enable the otherwise standard vendor boot-loader and kernel to work with the NDI FPGA examples.

4.1 ALTERA

The Altera U-Boot bootloader and Linux kernel are built from git repositories provided by altera-opensource on github. See RocketBoards.org for full details.

4.1.1 DIRECTORY STRUCTURE

The following Altera project subdirectories can be found in the `fpga_reference_design/linux_kernel` directory:

Directory	Target Development Board
agilex7	DK-SI-AGI027FA Agilex-7 SoC DevKit
a10-socdk	Arria-10 SoC Development Kit
SoCKit	Arrow/Terasic SoCKit

4.1.2 BUILD REQUIREMENTS

Building the Linux kernel and U-Boot bootloader requires an appropriate cross compiler for the target archiecture. On Debian and similar systems, this can be accomplished by installing the appropriate packages:

```
sudo apt-get install cpp-aarch64-linux-gnu
sudo apt-get install gcc-arm-linux-gnueabihf
```

Alternately an appropriate 3rd party cross compiler can be used, or a toolchain can be built directly from source.

4.1.3 BUILDING AN EXISTING PROJECT

The build instructions from Altera involve quite a few manual steps:

https://www.rocketboards.org/foswiki/Documentation/BuildingBootloaderCycloneVAndArria10
https://altera-fpga.github.io/rel-25.1/embedded-designs/agilex-7/f-series/soc/boot-examples/ug-linux-boot-agx7-soc/

These steps have mostly been automated via a Makefile, with some additional changes to customize the kernel as needed for the NDI Advanced SDK (enable UIO devices and create custom device-tree nodes).

The Makefile will checkout the source directories, patch and configure as required, then build the files required to boot and create a uSD image. Warnings about watchdog and timer drivers are expected while building U-Boot from the Altera released sources and can be ignored.

The configuration snippet `uio.fragment` enables the required uio kernel modules.

The patch file `0001-NewTek-Altera-NDI-device-tree-entries.patch` adds the required device-tree nodes for logic in the FPGA fabric. See the instructions from Altera for full details:

https://www.rocketboards.org/foswiki/Documentation/HOWTOCreateADeviceTree

4.1.3.1 NOTE:

If you follow the RocketBoards instructions for generating the Cyclone-V boot files, the FPGA will not be programmed by default. The provided Makefile builds a u-boot.scr file which is executed by the default Altera U-Boot bootcmd. This script programs the FPGA prior to loading the Linux kernel and device-tree.

4.2 XILINX

The following boards and NDI configurations are supported. For a full list of available targets run `make help` in the `fpga_reference_design/linux_kernel` directory.

Configuration	Contents
Arty-Z7-20-Enc	NDI encoder for the Digilent Arty Z7-20
Zybo-Z7-20-Enc	NDI encoder for the Digilent Zybo Z7-20
Zybo-Z7-20-Enc-Lite	NDI encoder for the Digilent Zybo Z7-20 with 16-bit SDRAM
Zybo-Z7-20-Dec	NDI decoder for the Digilent Zybo Z7-20
ZCU104-Enc	NDI encoder for the Xilinx ZCU104
ZCU104-Dec	NDI decoder for the Xilinx ZCU104

Xilinx offers at least two different approaches to generating the bootloaders, device tree, and Linux kernel.

- Open Source Linux (OSL) flow
- PetaLinux flow

4.2.1 OPEN SOURCE LINUX (OSL) FLOW

The OSL flow essentially builds each component required to boot the example design independently and then packages them together in a `BOOT.BIN` file along with a kernel, kernel modules, kernel headers, and a device tree. It offers the highest degree of transparency and control to the user and avoids the need to deal with the vagaries of alternate methods like Yocto or PetaLinux. It requires only that the user have installed the appropriate version of Vitis, which provides the ARM cross-compilers, XSCT, `bootgen`, and other supporting tools.

4.2.1.1 REQUIRED XILINX TOOLS

Building the Xilinx example designs requires that the user have installed the AMD Vitis Unified Software Platform version 2024.2. Xilinx has recently merged Vivado and Vitis together and they generally should be installed concurrently. Vitis provides example design source code, the SDK, bare-metal libraries, as well as the cross-compilers, XSCT, `bootgen` and other supporting tools.

Installation instructions, supported platforms, and other details can be found in “UG1742, Vitis Release Notes and Installation Guide”. After installation, users should prepare their environment by sourcing the appropriate settings script, found at `<install_dir>/Vitis/2024.2/settings64.sh` before attempting to build any of the example designs.

4.2.1.2 GENERATING EXAMPLE DESIGNS

The steps required for building the bootloaders and kernel are described to some degree on the Xilinx wiki and forums and have generally been automated via a top-level Makefile. To build one of the example designs, run `make all` with the following variables set accordingly:

- `XSA_BOARD` - The target board: `ZCU104`, `Zybo-Z7-20`, or `Arty-Z7-20`
- `XSA_BOARD_PROJ` - The design to build: `Enc`, `Dec`, or `Enc-lite` (Zybo only)

Note that case is significant and that the combination matches the project names in `../src/fpga` as well as the device tree snippets in `device-tree/`.

4.2.1.3 CONFIGURATION FILES

Each Xilinx board has a corresponding entry in `boards/` which contains configuration settings used by the make process. The appropriate file to use is determined by the value of the `XSA_BOARD` variable and determines the flags to use for compiling the bootloader, the kernel, and other settings. Refer to the Makefile and fragments in `mk/` for more details.

Each Xilinx project has a corresponding entry in `device-tree/` which contains a device tree include file that will be included in the device tree source and ultimately compiled during the build process. The appropriate file to use is determined by the value of the `XSA_BOARD_PROJ` variable. Refer to the Makefile and fragments in `mk/` for more details.

4.2.1.4 OUTPUT PRODUCTS

There are many ways to boot Xilinx devices, this discussion is limited to what is done here. The build process produces several artifacts required for booting the system:

- `BOOT.BIN` - The initial binary used to boot the system prior to loading the kernel
- `system.dtb` - The flattened device tree passed to the kernel by U-boot
- `Image` or `zImage` - The Linux kernel loaded by U-boot
- Kernel modules and headers

A BOOTLOADERS AND BOOT.BIN

The `BOOT.BIN` file created for the example designs contains the following content:

- First-stage bootloader (FSBL)
- U-boot ELF
- Minimal U-boot device tree (used by U-Boot before loading the kernel)
- Bitstream

Ultrascale+ devices also contain: - ARM Trusted Firmware (ATF) - Platform management unit (PMU) firmware

The origins of each component of the `BOOT.BIN` are the following:

- The FSBL source code is exported by XSCT using the contents of the associated Xilinx support archive. After export, the source code and linker script are cross compiled for the target platform. Build flags for the FSBL are controlled by variables set in the appropriate board specific file in the `boards/` directory. The FSBL ELF is staged for later packaging.
- The Makefile fetches the specified tag of the Xilinx U-boot fork, applies the defconfig from the file defined by `XSA_BOARD` and cross-compiles it for the target platform. The U-boot ELF and a minimal device tree are staged for later packaging. It is worth noting that the Xilinx platforms used for NDI Advanced SDK example designs all have some level of support by U-boot and the device tree that is created is sufficient to boot a kernel from the SD card.

- The bitstream is extracted from the provided Xilinx support archive (XSA) file and staged for later packaging.
- The PMU firmware (ZCU104 only) source code is exported by XSCT using the contents of the XSA file. The PMUFW is cross-compiled in the same was as the FSBL, with PMUFW specific build variables that can be useful for debugging. The user is referred to the Xilinx wiki and especially UG1137, the Zynq MPSoC Software Developers Guide, for more details regarding PMU use and configuration.
- The Makefile fetches the specified tag of the ARM trusted firmware (ATF) and builds it for the target platform. The `bl31.elf` it creates is then staged for later packaging.
- Once the FSBL and U-boot binaries, minimal U-boot device tree, bitstream, and PMUFW and ATF are staged, the appropriate `.bif` file is installed alongside them and `bootgen` is run construct the `BOOT.BIN` that will eventually be transferred to `/boot` of an SD card. An MD5 hash is computed for each component as well as the final product. See UG1283 for more information on the use of `bootgen` to create and dump `BOOT.BIN` files.
- Once `bootgen` has finished assembling the `BOOT.BIN` file, it is copied to a project specific location in the `output` directory.

B FLATTENED DEVICE TREE

Generating the device tree source code and the flattened device tree requires several steps:

- The Makefile fetches the indicated tag of the Xilinx device-tree source generator which is a large collection of Tcl scripts and board specific details for Xilinx IP and development boards. The DTS generator uses the XSA to infer IP settings and connections.
- The path to that repo is used together with the project XSA file to create a directory containing all the files required to build a device tree for the specific Vivado project.
- The files in the device tree source directory then undergo some modifications (see `mk/dtb.mk` for details) prior to compilation.
- The appropriate `.dtsi` file in `device-tree/` is included at this stage before the final device tree source is compiled into the flattened device tree.
- The flattened device tree and the top level source code are copied into the same project-specific location in the `output` directory that the `BOOT.BIN` was.

C KERNEL AND MODULES

The Linux kernel follows a much more familiar set of steps:

- The Makefile fetches the indicated tag of the Xilinx fork of the Linux kernel source tree. To save space and time, this is a limited clone of the tree.
- The kernel is built using an external build directory (using the `O=` option), which allows the same source tree to be used to build multiple architectures.
- Default kernel configurations are used for both ARM and ARM64 kernels.
- The kernel, modules, and headers are installed into `output/$(ARCH)/`.

4.2.1.5 REBUILDING EXAMPLE DESIGNS AFTER MODIFICATION

Rebuilding example designs after modifications to the FPGA projects is straightforward as long as a couple of caveats are followed:

- The project naming conventions need to remain the same in order for device tree include files to be properly included
- In principle, any new IP added that the kernel needs to be aware of will be automatically included by the device tree source generator. In practice, some manual modifications via the `.dtsi` file may be required.
- Run `make clean` which will remove the staging area and output directories but preserve the git repos that were downloaded to `extern/`. Then run `make all` with the `XSA_BOARD` and `XSA_BOARD_PROJ` set appropriately.

4.2.2 PETALINUX FLOW

The PetaLinux design flow is an alternative to the OSL flow that the NDI Advanced SDK uses. No PetaLinux projects are provided.

4.2.2.1 SUGGESTED STEPS FOR PETALINUX PROJECTS

The PetaLinux commands and recommended work flow have changed in recent revisions of the tools and the following steps should only be used as a guide. For more detailed information, including the most recent commands and arguments, refer to "UG1144 PetaLinux Tools Documentation: Reference Guide" and other documents, in particular the Xilinx wiki.

The general approach to building a PetaLinux project based on the NDI Advanced SDK would be the following:

- Export the hardware platform from Vivado and make sure that the bitstream is included in the exported Xilinx support archive (XSA)
- Create a PetaLinux project

```
$ petalinux-create -t project <project_name>
```

- Import the new hardware definition and configure the project

```
$ cd <project_name> $ petalinux-config --get-hw-description <path-to-vivado-project>
```

- Configure the project, kernel, U-boot, and rootfs

```
$ petalinux-config -c u-boot $ petalinux-config -c kernel $ petalinux-config -c rootfs
```

- Add device tree include files from `device-tree/` for the target platform to the PetaLinux project. This has typically been done by copying it to a location with the `project-meta` directory, but this may vary by version. If the design to be built includes additional IP or other modifications that affect the device tree, these need to be made as well, typically by hand.
- Build the project.

```
$ petalinux-build
```

- Package the system for deployment (note that there are likely many options that will need to be included for this command)

```
$ petalinux-package <options>
```

Once the project has been completed, boot images are typically located in the project tree at `images/linux`. The kernel and kernel modules will need to be extracted from the project along with the device tree and `BOOT.BIN` for transfer to an SD card. Various kernel images are usually generated during the build process. Kernel modules can be extracted from the rootfs tarball that is created during the build.

5 USD IMAGE BUILDER

Scripts to build uSD images for NDI demo platforms based on Debian can be found in the `fpga_reference_design/os_uSD` directory:

File	Description
`README.md`	This file
`CHANGELOG.md`	List of notable changes
`a10-socdk.conf`	Config file for Arria-10 SoC Development Kit
`agilex7.conf`	Config file for Agilex-7 SoC Development Kit
`sockit.conf`	Config file for SoCKit image
`zcu104.conf`	Config file for ZCU104
`zybo.conf`	Config file for Zybo-Z7-20 and Arty-Z7-20
`build.sh`	Main image build script
`common.sh`	Functions common to several scripts
`boot.sh`	Extracts boot files from a PetaLinux project
`debootstrap.sh`	Creates a Debian root filesystem from scratch
`second-stage.sh`	The debootstrap second-stage and rootfs customizations
`mk.uSD.sh`	Creates a uSD image from boot files and a rootfs
`part.txt`	Partion table for the uSD
`part.a2.txt`	Partion table for uSD images with Altera A2 partition
`update.*.tgz.sh`	Example to generate optional tgz files

The build scripts use a configuration file which provides details needed to create an image. The main details required are a pointer to the boot files and various architecture specific information (eg: the Debian architecture name and the qemu static binary to use for an emulated chroot environment). Various other settings may be tweaked as well, including the uSD target size, system host name, etc. Refer to the existing `\*.conf` files for details.

5.1 BUILD DEPENDENCIES

- `debootstrap`

  Used to create a root filesystem image. Currently using version 1.0.128nmu2 with Debian 13 (trixie). Many operating systems ship with an older version of debootstrap that is available through their package managers. It is recommended that a more recent version be installed from a preexisting .deb package. See Section D.3.2. Install debootstrap of the Debian GNU/Linux Installation Guide for more details.

- `qemu-static`

  Required to run debootstrap second-stage. Currently tested with version 4.2.1. Note that transparent emulation needs to be set up on the host system. See the QEMU documentation for more details on how to configure this.

5.2 BUILD AN IMAGE

To build an image from scratch, simply run the build.sh script with the desired configuration specified:

```
./build.sh zcu104.conf
```

5.3 REBUILD AN IMAGE

It is often not necessary to run all steps from scratch. Once an initial build has created the required bootfs and rootfs directories, each component of the uSD image may be updated independently. The components that makeup the uSD image are:

- `bootfs.<board>/`

  This directory contains the Xilinx boot files and linux kernel modules extracted from the PetaLinux project. This directory is created and updated by running the `boot.sh` script.

- `../linux_kernel/<board>/boot`

  This directory contains the Altera boot files and linux kernel modules. This directory is created and updated by running make in the `../linux_kernel/<board>` directory.

- `root.<deb_arch>.tgz`

  This file is manually generated and if present will be extracted to the root directory of the target file-system by the root user. This is currently used to install pre-compiled NDI example binaries to /usr/local/bin and the required shared libraries to /usr/local/lib.

  This file is processed by the `second-stage.sh` script, so any changes require re-running `debootstrap.sh` or manually applying the changes to the rootfs directory.

  Note this file should include full paths preceded by a leading ./ eg: `./usr/local/lib/file.so`. Below is one example of how to properly create this file:

  ```
  # Start at the root directory
  cd /

  # Create a tar archive with a leading ./ and put it in /tmp
  tar -czvf /tmp/root.armhf.tgz ./usr/local/bin/*
  ```

- `user.<deb_arch>.tgz`

  This file is similar to the `root.<deb_arch>.tgz` file, above, except it is extracted into the default user's home directory by the default user. This file is used to install the HDMI start and stop scripts for the ZCU104, and is unused by the Zybo-Z7-20 example. Place any files that need to be owned by the default user and thus cannot be placed in the root archive (since the default username, uid/gid, and home directory may change) into this archive.

  This file is processed by the `second-stage.sh` script, so any changes require re-running `debootstrap.sh` or manually applying the changes to the rootfs directory.

- `rootfs.<deb_arch>/`

  This directory contains a stock Debian install generated by debootstrap along with some modifications required to make a usable image (eg: create /etc/fstab and enable networking) and tweaks required for this example (eg: build and install the libuio package). This directory is generated by running the `debootstrap.sh` script, while most of the customizations to the rootfs occur in the `second-stage.sh` script.

  Running debootstrap.sh deletes any existing rootfs and recreates it from scratch, which is a lengthy process and does not allow for modifications other than editing the `second-stage.sh` script. Once created, however, the rootfs directory may be edited manually, just be careful with file-system permissions and user ids. It is also possible to chroot to the rootfs and execute native commands, eg:

  ```
  # Setup shell variables for our configuration
  source zcu104.conf

  # Open a native shell in the rootfs
  sudo LANG=C.UTF-8 chroot $ROOTFS /bin/bash
  ```

Once any desired changes have been made to the above components, a new image can be created by running:

```
sudo ./mk.uSD.sh <config>.conf
```

6 CHANGELOG

Versions: Major.Minor.Patch

Changelogs are synchronized for Major.Minor version numbers, but each project has it's own patch version (eg: src/fpga might be version 1.1.2, and src/cpp could be 1.1.4).

[6.3.0] - 2025-11-21

Added

- Added generics to NDI IP cores to enable/disable features
- Agilex-7 encode example design

Changed

- Xilinx NDI IP cores modified to reduce BRAM usage
- Xilinx kernel and boot loader build method
- Zynq 7000 projects updated to Vivado 2024.2

[6.1.0] - 2024-11-22

Added

- Virtual PTZ support to ndi_encode application

Changed

- NDI Advanced SDK uses the `libndi_advanced` library now
- SD card images updated with latest NDI library and example applications

[6.1.0-rc1] - 2024-09-13

Added

- Synchronize major version number with software SDK version
- Encoder support for planar alpha
- Support for new packed and semi-planar video formats
- Support for 16-bit video (11-bits passed through SpeedHQ codec)
- Support for 64-bit addressing in raw audio/video input and output logic

[1.5.4] - 2024-01-24

Added

- Example project for the Kria KV260 development board

Fixed

- A couple bugs in the clear-text FPGA logic

[1.5.3] - 2023-08-25

Changed

- Xilinx projects updated to Vivado 2022.1
- Zynq 7000 HDMI Rx Logic improved
- SoCKit projects deprecated as boards are no longer available for purchase

[1.5.0] - 2022-10-09

Added

- Alpha support (interleaved and planar) added to NDI_Dec
- Example projects targeting the Arria-10 SoC Development Kit

Changed

- Updates for NDI v5.x

[1.4.0] - 2020-03-25

Added

- NDI Decode support

Changed

- Updates for NDI v4.5
- Xilinx & Petalinux projects updated to 2019.2

[1.3.0] - 2019-07-08

Changed

- Updates for NDI v4.0
- Refactoring to support both encode and decode
- Updates to device-tree layout and uio device names
- Zybo uSD image modified to include all three examples (Encode, Encode-Lite, Decode)

[1.2.1] - 2018-11-17

Added

- Zybo-Z7-20-Lite example design (16-bit SDRAM interface with 2 encoder cores)

[1.2.0] - 2018-10-03

Added

- Auto-detect video format
- Audio is now working

[1.1.1] - 2018-09-25

Changed

- Sub-project zip files now provided in expanded form
- Restructured project directories

[1.1.0] - 2018-08-31

Added

- Support new hardware version register

Changed

- Updated NDI_Demo hardware project files
- Updated PetaLinux files

[1.0.0] - 2018-07-13

Added

- Initial version

Changelog Hints & Details

- Changelog format
- Semantic Versioning

6.1 FPGA PROJECTS CHANGELOG

[6.3.0] - 2025-11-21

Changed

- Add generics to enable/disable features in NDI IP cores
- Add error bits to NDI IP cores indicating unsupported format requested
- Reduced BRAM usage in Xilinx NDI IP cores (distributed RAM now used where appropraite)
- Update Zybo and Arty projects to Vivado 2024.2

Added

- Agilex-7 encoder project

[6.1.0] - 2024-11-22

Changed

- No changes from 6.1.0-rc1

[6.1.0-rc1] - 2024-09-13

Changed

- Reset version numbering to align with software SDK
- Memory simulation files now explicitly labeled as encoder or decoder specific
- Many changes to decoder and encoder cores (see NDI 6.1 hardware changes for details)
- Upgraded the message severity in Xilinx projects for missing memory initialization files (these are reported as errors now)
- Rename output files to be consistent with top-level project name

Added

- Encoder support for planar alpha
- Support for new packed and semi-planar video formats
- Support for 16-bit video (11-bits passed through SpeedHQ codecs)
- Add 6.1 decoder and encoder encrypted IP and memory initialization files to all example designs
- Support for 64-bit addressing in raw audio/video input and output logic

Fixed

- NDI_Enc change handling of VID_BURST_WIDTH generic so only legal Avalon burst lengths are generated. Previous logic used b”0000” to represent a 16 word burst when the correct value should be b”1_0000”, which becomes b”1111” when converted into an AXI arlen value
- NDI_Enc VID_BURST_WIDTH generic set to 5 for Zynq 7000 projects

[1.5.4] - 2024-01-24

Added

- Example project for the Kria KV260 development board

Fixed

- Local reset logic in Vid_In, Vid_Out, and Vid_Track was ignoring rst
- Bug in Avl_Axi_Wr could cause bus lockup under some conditions

[1.5.3] - 2023-08-25

Changed

- Xilinx projects updated to Vivado 2022.1
- Switch to performance optimized synthesis and implementation strategies to ease timing closure
- Merged Hamsterworks HDMI handling with Digilent PHY
- Update Xilinx encryption key to xilinxt_2019_02

[1.5.2] - 2023-01-12

Changed

- Update SoCKit-Dec project to Quartus 22.1

[1.5.1] - 2022-09-19

Added

- Initial example design for Arria-10 SoC Devkit
- NDI_Dec now supports writing planar alpha when PLANAR_ALPHA generic is true
- PLANAR_ALPHA generic added to disable planar alpha logic if not needed

Fixed

- Inferred multiplier output register did not map to DSP block in Arria-10

Changed

- Update generics for NDI_Enc to support different read and write bus parameters
- Update SoCKit-Dec project to Quartus 20.1.1
- Update SoCKit-Dec software files to SoC EDS 20.1 for U-Boot socfpga_2021.10

[1.4.9] - 2022-07-12

Fixed

- Updated cache and user bits in Avl_Axi_Wr.vhd to address cache coherency issues on the Zynq 7000 Encoder example causing corrupted bitstreams

[1.4.8] - 2022-01-10

Added

- Semantic versioning to NDI cores version register
- wr_alpha control bit to NDI_Dec to enable writing alpha data

[1.4.7] - 2021-11-24

Added

- Disable bit to Vid_Out.vhd
- Support for variable counter widths in Vid_Track.vhd

Fixed

- Encode_x4 updated to properly merge audio input data with fewer than 4 cores (previous fix was incorrect)
- Parallel audio left/right data swapped in Aud_In.vhd

[1.4.6] - 2021-11-04

Added

- Missing dtsi files for Altera kernel

[1.4.5] - 2021-03-30

Fixed

- Encode_x4 updated to properly merge audio input data with fewer than 4 cores
- Update encode projects to read back zeros when accessing decoder addresses

Changed

- Routed FPGA SDRAM status signals to LEDs for SoCKit design

[1.4.4] - 2021-03-02

Added
- Initial version of Arty-Z7-20-Enc

[1.4.3] - 2020-06-02

Fixed
- Bug in Preview logic when set to divide by 2 in wide mode (720p on the ZCU104)
- 4:2:2 to 4:4:4 conversion logic in DVI_Tx.vhd

[1.4.2] - 2020-05-20

Fixed
- Problem with Altera specific Decode logic

[1.4.1] - 2020-03-25

Fixed
- Audio Output register address for ZCU104 Decode project

[1.4.0] - 2020-03-20

Changed
- Xilinx projects updated to Vivado 2019.2
- Intel (Altera) project updated to Quartus-Lite 19.1
- NDI Decode support

[1.3.4] - 2020-03-17

Added
- Initial version with audio output

Fixed
- Updated FIFO and command reset logic in Vid_Out

[1.3.3] - 2019-10-07

Fixed
- Fixed quantized coefficient rounding in FPGA Encode logic so it matches the software

[1.3.2] - 2019-09-20

Changed
- NDI Decoder updated for Altera DSP blocks

[1.3.1] - 2019-08-29

Added
- ZCU104 decode reference design
- Decode and output support for 4Kp60 4:2:0 video

Changed
- Update FPGA logic to improve timings
- 4 Macroblock burst mode added to NDI_Dec to improve SDRAM efficiency

[1.3.0] - 2019-07-08

Added
- Initial version with NDI Decode

Changed
- Added support for targeting all Encoder cores with a single write
- Updated Altera licenses

[1.2.2] - 2019-04-04

Added
- Overview of Encoder core: NDI_SoC+FPGA_Encoder.pdf

Changed
- Updated block diagram and migrated it into Encoder overview pdf file

Fixed
- Update Avl_Axi_Wr.vhd to avoid bus lockup under rare conditions
- Fix case mis-match with local.xdc file in Zybo-Z7-20-Lite project file

[1.2.1] - 2018-11-17

Added
- Zybo-Z7-20-Lite example design (16-bit SDRAM interface with 2 encoder cores)

Fixed
- Preview filter was not passing locked bit to output
- Missing signal declaration in “Hamsterworks” HDMI audio logic
- Encode_x4 updated to properly support fewer than 4 cores

[1.2.0] - 2018-09-30

Added
- Audio support
- Video tracking and auto-format detection

Changed
- Renamed ZCU104 project directory

[1.1.2] - 2018-09-21

Added
- Altera IP core and license file (example design coming soon!)
- Hardware export directories
- Compiled bit files

Fixed
- Slightly improved NDI encoder efficiency to improve performance at 4Kp60 when using a 200 MHz clock
- Fix issue when switching between SD and HD modes
- Fix wedging issue with specific memory latency and wait state patterns

[1.1.1] - 2018-09-13

Added
- HDMI embedded audio extraction
- Digilent dvi2rgb added as an alternative video input

Fixed
- RGB to YCbCr color space conversion (B and R coefficients were swapped)

[1.1.0] - 2018-08-31

Added

- HDMI Input logic for Zybo platform, based on open-source code from Mike Field hamster@snap.net.nz. Thanks Mike!!!
- Version register including platform specifier
- PS block design: axi_gpio to interface to PL LEDs and push-button switches
- PS block design: axi_iic for audio codec I2C bus (Zybo)
- Audio input logic added to Zybo project (not yet tested)
- Details on compiling HDMI Rx code for the Cortex-R5 on the ZCU104
- Automated zip file package builds thanks to git archive (git ROCKS! :))

Fixed

- Critical warning regarding VIDEO_CLK when building ZCU104 project
- Issues caused by running on platforms with both 128-bit (ZCU104) and 64-bit (Zybo-Z7) interfaces to SDRAM.

Changed

- README.txt switched to markdown format and renamed to README.md
- General code cleanup and removal of unnecessary files, comments, and deprecated code.
- Added actual purging logic to VidIn, rather than just resetting the FIFO when VSYNC is active.

[1.0.2] - 2018-08-07

Changed

- Add Build Dependencies section to the README file indicating Digilent board files must be installed to properly build the example Zybo project and an HDMI license is required for the ZCU104.

Fixed

- Deprecated file “NDI_Pkg.vhd” removed from Zybo-Z7-20 project file.

[1.0.1] - 2018-07-26

Added

- Changelog.md

Fixed

- Deprecated file “NDI_Pkg.vhd” removed from zynqmp.NDI project file.

[1.0.0] - 2018-07-13

Added

- Initial version

6.2 C++ APPLICATION CODE CHANGELOG

[6.3.0] - 2025-11-21

Added

- Support for new hardware error bits when requesting an unsupported format
- Support for hardware configurations that were compiled without planar video or 16-bit format support

Changed

- The `NewTek_Reserved` and `NewTek_Video` reserved memory regions are looked up in the device tree by alias first and then, if absent, looked for directly in `/sys`.

[6.1.x] - 2024-12-31

Added

- Support for playback of packed output video formats - 4:2:2 UYVY, YUYV, Y216, and UYVW formats - 4:2:0 420 and 420W formats
- Support for luminance only playback of semi-planar 4:2:2 formats

Changed

- The `ndi_decode` appliation now supports strings instead of requiring numerical arguments to the `-o` option to indicate the output video format
- Playback of 16-bit formats is truncated to 8-bit before being sent to the HDMI output interface

[6.1.0] - 2024-11-22

Added

- Virtual PTZ support to ndi_encode application

Changed

- NDI Advanced SDK uses the `libndi_advanced` library now

[6.1.0-rc1] - 2024-09-13

Changed

- Reset version numbering to align with software SDK

Added

- Encoder support for planar alpha
- Support for new packed and semi-planar video formats
- Support for 16-bit video (11-bits passed through SpeedHQ codec)
- Additional decoder application features: - Burst vs. no-burst mode SDRAM transfers - New video capture class and command line options for capturing video frames - Output video format can be selected at run-time (default to 8-bit UYVY)

[1.5.1] - 2023-01-13

Added

- Support for Arria-10 SoC Developemnt Kit platform

Changed

- Disable CPU intensive protocols on lower-end platforms

[1.5.0] - 2022-08-05

Changed

- Updates for NDI 5.x - Remove frame copy functions in video_compress.cpp - Pass scatter gather list and let NDI SDK perform the copy - Update examples with new NDI Vendor ID format

****Fixed**

- Changed pattern generator line stride to avoid issues in 4:2:0 mode

[1.4.2] - 2022-07-12

Fixed

- SSM2603 Line Input was not working

[1.4.1] - 2020-03-27

Changed

- Switch ZCU104 to I2S serial audio output format

[1.4.0] - 2020-03-25

Changed

- Updates for NDI v4.5
- NDI Decode support

[1.3.3] - 2020-02-05

Changed

- Switched to NDI SDK function to generate next Q value for video encoding
- Updated single-character debug output so each character is unique

Fixed

- Addressed the various `FIXME` comments in the ndi_encode application code

[1.3.2] - 2020-01-07

Added

- Support for SoCKit platform

[1.3.1] - 2019-09-26

Changed

- Code migrated to separate sub-directories for each target application
- Makefile updates to better manage auto dependency generation

[1.3.0] - 2019-07-08

Changed

- Updates for NDI v4.0
- Refactoring to support both encode and decode

[1.2.2] - 2018-11-17

Added

- Support for changing reserved memory region size in device-tree
- Support for fewer than 4 encoder cores

Fixed

- Unused variable warning when performing a release build

[1.2.1] - 2018-10-10

Added

- Support for changing frame rate in pattern generator mode

Changed

- Zybo audio codec setup migrated to ndi_send application

Fixed

- Pattern Generator is working again
- Pattern Generator 4K video mode is working
- Video tracking thread was consuming excessive CPU cycles

[1.2.0] - 2018-10-03

Added

- Auto-detect video format
- Audio is now working

[1.1.1] - 2018-08-25

Changed

- Sub-project zip files now provided in expanded form
- Restructured project directories
- ndi_device renamed ndi_send
- Generate proper timestamps

Fixed

- Issue with potential use after free related to addition of copy thread

[1.1.0] - 2018-08-31

Added

- ndi_reg utility
- Support new hardware version register

Changed

- General cleanup and removal of deprecated/commented code
- Pull details on raw video memory from device tree if available
- Add copy thread to video path

[1.0.0] - 2018-07-13

Added

- Initial version

6.3 LINUX KERNEL AND BOOT LOADER CHANGELOG

[6.3.0] - 2025-11-21

Added

- Agilex-7 kernel and boot-loader

Changed

- Updated Arty and Zybo targets to use Xilinx 2024.2 tools
- Updated ZCU104 targets to use Xilinx 2022.1 tools
- Makefile updated to build targets directly from Xilinx repositories instead of using PetaLinux
- Removed Petalinux projects and Makefile logic

[6.1.0] - 2024-11-22

Changed

- No changes from 6.1.0-rc1

[6.1.0-rc1] - 2024-09-13

Changed

- Reset version numbering to align with software SDK
- Update Makefile to use new exported FPGA output file names

[1.5.3] - 2023-08-25

Changed

- Migrated all PetaLinux projects from 2019.2 to 2022.1
- Kernel version for Xilinx projects changed to 5.15.19
- All Zynq-7000 and UltraScale+ projects are automatically generated via make and bash scripts.
- Petalinux project configuration and device tree modifications automatically incorporated

[1.5.2] - 2023-01-12

Added

- SoCKit updated to Quartus 22.1

[1.5.1] - 2022-09-22

Added

- Initial support for Arria-10 SoC Development Kit
- ndiname device-tree node for use as NDI machinename

Changed

- Removed deprecated Petalinux 2018.1 projects
- Move reserved memory regions in Zybo-Z7-20-Lite device-tree

[1.4.2] - 2021-03-08

Added

- Initial support for Arty-Z7-20

[1.4.1] - 2020-03-27

Fixed

- R5 remoteproc device-tree entries for ZCU104-Dec project were broken

Added

- Added prebuilt boot files required to generate uSD images

[1.4.0] - 2020-03-25

Changed

- Updates for NDI v4.5
- Petalniux projects updated to 2019.2
- NDI Decode support

[1.3.3] - 2020-03-08

Added

- Petalinux 2019.2 project for ZCU104 ndi_encode

[1.3.2] - 2020-01-07

Changed

- Modified Zybo boot arguments to support new partition layout
- Updated pre-compiled files with latest FPGA bit files

Fixed

- Added missing pre-comiled U-Boot files for ZCU104 projects

[1.3.1] - 2019-07-11

Fixed

- Updated pre-compiled files to allow update of the various FPGA bit files (NDI Encode, Encode-Lite, and Decode) without rebuilding from scratch

[1.3.0] - 2019-07-01

Added
- Updates for NDI v4.0

Changed
- Updated system-user.dtsi files to support NDI Decode
- Updated pre-compiled files

[1.2.1] - 2018-11-15

Fixed
- Missing dash in `--get-hw-description` petalinux command

[1.2.0] - 2018-09-30

Added
- Added pre-compiled boot files, allowing update of the FPGA bit file without recompiling PetaLinux from scratch
- Added tracking logic to system-user.dtsi

Changed
- Tally LEDs default to on at boot instead of the heartbeat trigger
- Unused LEDs set to cpu and uSD trigger

[1.1.1] - 2018-09-24

Changed
- Update boot.sh scripts to reference bit file in `src/fpga/<project>` directories

Removed
- FPGA bit files (migrated to `src/fpga/`)
- Hardware export directories (migrated to `src/fpga/`)

[1.1.0] - 2018-09-05 Added : Hardware export sdk directories from Vivado

Changed
- Projects updated with latest FPGA hardware export
- Device-tree entries added for tally LEDs

[1.0.0] - 2018-08-17

Added
- Initial version

6.4 USD IMAGE BUILDER CHANGELOG

[6.3.0] - 2025-11-21

Changed
- Switched root filesystem from Debian 12 (bookworm) to Debian 13 (trixie)
- Update prebuilt tarballs with current NDI library and applications
- Create zip file after generating img.xz, bmap, and md5 files
- Output product location can now be specified with `LOCAL_TMP`
- Update Xilinx configurations with new locations for kernel and bootloader

Added
- Agilex-7 configuration

[6.1.0] - 2024-11-22

Changed

- Update prebuilt tarballs with current NDI library and applications

[6.1.0-rc1] - 2024-09-13

Changed

- Reset version numbering to align with software SDK
- Update prebuilt tarballs with current NDI library and applications

[1.5.3] - 2023-08-25

Changed

- Switched root filesystem from Debian 11 (bullseye) to Debian 12 (bookworm)
- Build updates for Petalinux 2022.1
- Using extlinux.conf instead of boot.scr due to changes with newer versions of the Xilinx tools
- Debootstrap now requires transparent emulation via QEMU

[1.5.1] - 2022-09-24

Added

- Image configuration for Arria-10 SoC Development Kit

Changed

- Update from Debian stretch to Bullseye
- Altera boot files updated to match current U-Boot generated files

[1.4.0] - 2020-03-25

Added

- Updates for NDI v4.5
- Composite uSD image supporting both Xilnix Digilent Zybo-Z7 and Intel/Altera Terasic SoCKit boards using the same uSD image
- NDI Decode support

[1.3.1] - 2020-01-07

Changed

- Change partition layout for armhf configurations to allow dual booting of Zybo-Z7-20 and SoCKit boards with the same uSD image
- Switch back to upstream head for libuio source code

[1.3.0] - 2019-07-07

Added

- Updates for NDI v4.0

Changed

- Allow for multiple boot filesets in uSD image generation
- Switch libuio repository used to build from source - Upstream broke uio nodes with no map entry

[1.1.1] - 2018-09-25

Added

- Resize root partition on first boot

Changed

- Updated for new directory structure

[1.1.0] - 2018-09-09

Added

- CHANGELOG.md
- build.sh script to do (mostly) everything to make an image
- update.tgz.sh

Changed

- Switched to using *.conf files for image details
- Factored general setup into common.sh
- Lots of code cleanup

[1.0.0] - 2018-07-13

Added

- Initial version